Comparative PCA Methods Analysis

Mini Project for DS3294: Data Science Practice

Sadekar Suchet Samir Debagnik Das Anshita Singh Aurokrupa Sahoo

April 14, 2026

Abstract

Principal Component Analysis (PCA) is one of the most widely used techniques in data science for dimensionality reduction and feature extraction. However, different algorithmic implementations of PCA carry significantly different computational and memory costs, and their suitability varies with dataset size, dimensionality, and data structure. This report presents a systematic, evidence-based comparison of six PCA implementations: Eigendecomposition of the Covariance Matrix, Singular Value Decomposition (SVD), Kernel PCA, Randomized PCA, Incremental PCA, and Sparse PCA. Each method is implemented from first principles, verified against Scikit-learn reference implementations, and evaluated in terms of theoretical complexity, empirical runtime and memory usage, numerical stability under noise, and scaling behaviour across synthetic datasets of controlled size and dimensionality. The results provide practical guidance on selecting the appropriate PCA technique for large-scale data analysis.

1 Introduction

Principal Component Analysis (PCA) is a multivariate statistical method that transforms high-dimensional data into a smaller set of uncorrelated variables called principal components [2]. These components are linear combinations of the original features chosen so that the first component captures the greatest variance in the data, the second captures the next greatest variance while remaining orthogonal to the first, and so on. Dimensionality reduction is achieved by retaining only the top k components, projecting the data from d dimensions down to k and discarding the least informative variation.

Despite its elegance and widespread applicability, PCA faces significant computational challenges at scale. The two classical approaches — eigenvalue decomposition (EVD) of the covariance matrix and singular value decomposition (SVD) of the centred data matrix — require the entire dataset to fit in memory and scale poorly with either the number of observations n or the number of features d [2]. Furthermore, standard PCA is inherently linear, and cannot capture non-linear structure in data lying on curved manifolds or other complex geometric configurations. These limitations have motivated the development of several alternative PCA methods, each offering different computational and representational trade-offs.

This project investigates six PCA implementations: PCA by Eigendecomposition of the Covariance Matrix, PCA by SVD, Kernel PCA, Randomized PCA, Incremental PCA, and Sparse PCA. Each method is implemented from first principles in Python and evaluated against Scikit-learn reference implementations [6]. The central aim is to analyse systematically how the computational and memory complexity of each method scales with dataset size and dimensionality, to identify the regimes in which each method is preferable, and to provide practical guidance on selecting the appropriate PCA technique for large-scale data analysis.

To support controlled experimentation, synthetic datasets with configurable numbers of observations and features are generated, covering correlated, uncorrelated, low-rank, and non-linear

data distributions. Each implementation is benchmarked for runtime and memory usage, assessed for numerical stability under noise, and compared in terms of metrics such as reconstruction error and explained variance.

2 Related Work

PCA was first proposed by Pearson [5] and later formalised by Hotelling [4]. The method has since become one of the cornerstones of exploratory data analysis and unsupervised machine learning. A comprehensive treatment of PCA — including its definition, geometry, biplot interpretation, and variants — is provided by Greenacre et al. [2], who also discuss the practical limitations of standard EVD and SVD for large datasets and review incremental and randomized approaches.

The computational bottleneck of standard PCA for large matrices has motivated several lines of work. Halko et al. [3] proposed randomized algorithms for low-rank matrix approximation that efficiently estimate the top-k singular vectors without computing a full SVD, achieving substantial speedups for large datasets where only a small number of components are required. Incremental approaches to SVD and PCA, suited to streaming or memory-limited settings, have been developed by several authors [1, 7].

Kernel PCA was introduced by Schölkopf et al. [8] as a nonlinear generalisation of standard PCA. By applying the kernel trick, Kernel PCA performs linear PCA in a high-dimensional feature space implicitly defined by a kernel function, enabling the extraction of non-linear principal components without explicitly computing the feature map. A known limitation is its $O(N^2)$ memory cost and $O(N^3)$ time complexity arising from the $N \times N$ kernel matrix. Zheng et al. [9] proposed an improved algorithm that partitions the dataset into subsets and solves a reduced eigenvalue problem on a compressed kernel matrix, substantially reducing both time and memory requirements while preserving accuracy.

Sparse PCA, which produces principal components with sparse loadings via an L1 penalty, was formalised by Zou et al. [10] and has been applied extensively to high-dimensional biological data where interpretability of the components is important.

3 Our Contribution

This work contributes a unified, systematic empirical comparison of six PCA methods implemented from first principles. While individual methods have been studied in isolation in the literature, a direct side-by-side comparison under controlled experimental conditions, with consistent benchmarking methodology and identical synthetic datasets, is less common. Specifically, our contributions are:

- First-principles Python implementations of six PCA methods, each verified against Scikit-learn reference implementations.
- A shared benchmarking framework for measuring runtime and peak memory usage consistently across all methods.
- Synthetic dataset generators supporting correlated, uncorrelated, low-rank, and non-linear data distributions with configurable size and dimensionality.
- A systematic analysis of how each method scales with n (number of observations) and d (number of features), including identification of the regimes in which the standard approach becomes impractical.
- A numerical stability analysis that evaluates the sensitivity to input noise.

4 Methods

4.1 Synthetic Data Generation

Controlled experimentation requires datasets where the number of observations n and number of features d can be varied independently. Four types of synthetic datasets are generated.

Correlated data is produced by sampling from a multivariate Gaussian distribution with a specified covariance structure, ensuring that features exhibit known linear correlations.

Uncorrelated data is sampled from a diagonal covariance matrix so that features are independent.

Low-rank data is generated by constructing a data matrix as a product of low-rank factors plus Gaussian noise, producing data that lies approximately on a low-dimensional subspace.

Non-linear data is generated as concentric hyperspherical shells in two or three dimensions, providing a case where linear PCA cannot recover the true low-dimensional structure.

Each generator accepts parameters for the number of samples n, number of features d, noise level σ, and a random seed for reproducibility.

4.2 PCA by Eigendecomposition of the Covariance Matrix

The classical approach to PCA computes the covariance matrix of the centred data and performs eigendecomposition to obtain the principal component directions.

Given a data matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ with zero-mean columns, the covariance matrix is:

$$\mathbf{C} = \frac{1}{n-1}\mathbf{X}^\top \mathbf{X} \tag{1}$$

The principal components are the eigenvectors $\mathbf{v}_i$ satisfying $\mathbf{C}\mathbf{v}_i = \lambda_i \mathbf{v}_i$, where λ_i are the corresponding eigenvalues. Eigenvectors are sorted in descending order of eigenvalue, and the data is projected as $\mathbf{X}_{\text{reduced}} = \mathbf{X}\mathbf{V}_k$, where $\mathbf{V}_k$ contains the top k eigenvectors.

Complexity: Time $O(nd^2 + d^3)$; Space $O(d^2)$ for the covariance matrix.

4.3 PCA by Singular Value Decomposition

SVD decomposes the centred data matrix directly, avoiding explicit computation of the covariance matrix and offering improved numerical stability:

$$\mathbf{X}_c = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \tag{2}$$

where $\mathbf{U} \in \mathbb{R}^{n \times n}$ and $\mathbf{V} \in \mathbb{R}^{d \times d}$ are orthogonal matrices of left and right singular vectors respectively, and $\mathbf{\Sigma}$ is a diagonal matrix of singular values σ_i. The right singular vectors $\mathbf{V}$ are identical to the eigenvectors of the covariance matrix, with the relationship $\lambda_i = \sigma_i^2/(n-1)$ [2]. The reduced representation using the top k components is $\mathbf{X}_{\text{reduced}} = \mathbf{U}_k\mathbf{\Sigma}_k$.

Complexity: Time $O(\min(n,d)^2 \cdot \max(n,d))$; Space $O(nd)$.

4.4 Kernel PCA

Standard PCA is restricted to identifying linear structure. When the underlying data distribution is non-linear (for example, concentric shells or spiral manifolds) linear PCA fails to recover meaningful low-dimensional representations. Kernel PCA, introduced by Schölkopf et al. [8], addresses this by implicitly mapping the data into a high-dimensional feature space using a kernel function, then performing linear PCA in that space.

Mathematical Formulation

Let $\Phi : \mathbb{R}^d \to F$ denote an implicit nonlinear mapping from input space into a feature space F, which may have arbitrarily high dimensionality. The covariance matrix in F is:

$$\tilde{\mathbf{C}} = \frac{1}{N} \sum_{j=1}^{N} \Phi(\mathbf{x}_j)\Phi(\mathbf{x}_j)^\top \tag{3}$$

All solutions of the eigenvalue problem $\lambda \mathbf{V} = \tilde{\mathbf{C}}\mathbf{V}$ lie in the span of $\Phi(\mathbf{x}_1), \ldots, \Phi(\mathbf{x}_N)$, so we may write $\mathbf{V} = \sum_{i=1}^{N} \alpha_i \Phi(\mathbf{x}_i)$. This leads to the equivalent eigenvalue problem on the $N \times N$ kernel matrix $\mathbf{K}$:

$$N\lambda\boldsymbol{\alpha} = \mathbf{K}\boldsymbol{\alpha} \tag{4}$$

where entries of $\mathbf{K}$ are defined by a kernel function $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j) = \langle \Phi(\mathbf{x}_i), \Phi(\mathbf{x}_j) \rangle$. The kernel function computes dot products in F without explicitly evaluating Φ — this is the kernel trick. In this implementation the RBF kernel is used:

$$k(\mathbf{x}, \mathbf{y}) = \exp\left(-\gamma \|\mathbf{x} - \mathbf{y}\|^2\right) \tag{5}$$

where $\gamma > 0$ controls the width of the kernel. Since the mapped data cannot in general be centred directly in F, the kernel matrix is centred analytically:

$$\mathbf{K}_c = \mathbf{K} - \mathbf{1}_N\mathbf{K} - \mathbf{K}\mathbf{1}_N + \mathbf{1}_N\mathbf{K}\mathbf{1}_N \tag{6}$$

where $\mathbf{1}_N$ is the $N \times N$ matrix with all entries $1/N$. The projection of a test point $\mathbf{x}$ onto the k-th principal component in F is:

$$\left(\mathbf{V}^k \cdot \Phi(\mathbf{x})\right) = \sum_{i=1}^{N} \alpha_i^k \, k(\mathbf{x}_i, \mathbf{x}) \tag{7}$$

Implementation

The baseline implementation follows Schölkopf et al. [8] directly. Pairwise squared Euclidean distances are computed using `scipy.spatial.distance`, the RBF kernel matrix is constructed, analytically centred using Eq. 6, and the eigenvalue problem is solved using `scipy.linalg.eigh`. Eigenvectors are sorted in descending order of eigenvalue, negative eigenvalues arising from numerical noise are clipped to zero, and the top k components are returned scaled by the square root of their eigenvalues to produce projections comparable with Scikit-learn's `KernelPCA`.

Computational Complexity

The standard Kernel PCA algorithm has time complexity $O(N^2 d)$ for computing the kernel matrix and $O(N^3)$ for the eigendecomposition, giving an overall complexity of $O(N^3)$ for large N. The space complexity is $O(N^2)$ for storing the full kernel matrix. This quadratic memory requirement is the primary practical bottleneck: for $N = 10{,}000$ points, the kernel matrix requires approximately 800 MB, making the standard algorithm infeasible for large datasets.

Algorithmic Improvement

To address the scalability limitations of standard Kernel PCA, Zheng et al. [9] proposed an improved algorithm that avoids storing the full $N \times N$ kernel matrix by partitioning the dataset into M subsets and solving a reduced eigenvalue problem on a compressed kernel matrix of size $\tilde{N} \times \tilde{N}$, where $\tilde{N} = M \cdot p \ll N$.

We tried to adopt this approach to improve our Kernel PCA algorithm, but were not successful due to time constraints in completing the project.

Correctness Verification

The baseline implementation is verified against Scikit-learn's `KernelPCA` with an RBF kernel on non-linear synthetic dataset. Agreement is assessed using mean squared difference between projections, and difference in explained variance.

Numerical Stability Analysis

Numerical stability is assessed in terms of sensitivity to input noise: Gaussian noise of increasing standard deviation σ is added to a clean dataset, and the change in explained variance ratio and projections between the clean and noisy solutions is recorded.

4.5 Randomized PCA

Randomized PCA [3] uses random projections to efficiently estimate the top-k singular vectors without computing a full SVD, making it well-suited to large datasets where only a small number of components are needed.

A random projection matrix $\mathbf{\Omega} \in \mathbb{R}^{d\times(k+p)}$ is drawn from a standard Gaussian distribution, where p is an oversampling parameter. The data is projected as $\mathbf{Y} = \mathbf{X}_c\mathbf{\Omega}$, and an orthonormal basis $\mathbf{Q}$ for the column space of $\mathbf{Y}$ is obtained via QR decomposition. The data is then projected onto this basis: $\mathbf{B} = \mathbf{Q}^\top\mathbf{X}_c$, and a standard SVD is performed on the smaller matrix $\mathbf{B}$. The left singular vectors of $\mathbf{X}_c$ are recovered as $\mathbf{U} = \mathbf{Q}\mathbf{U}_B$.

Complexity: Time $O(ndk)$; Space $O(nk + dk)$. Substantially faster than full SVD for $k \ll \min(n, d)$, trading a small approximation error for a large computational gain.

4.6 Incremental PCA

Incremental PCA processes data in mini-batches of size b, updating the principal components as each batch arrives. This makes it suitable for datasets too large to fit in memory and for streaming data scenarios.

For each batch $\mathbf{X}_{\text{batch}}$, the running mean is updated and the batch is centred. The previous component matrix is augmented with the new batch data and a truncated SVD is performed on the augmented matrix, retaining only the top k components for the next iteration.

Complexity: Time $O(bdk)$ per batch, $O(ndk/b)$ total; Space $O(bd + kd)$, constant in n.

4.7 Sparse PCA

Sparse PCA [10] produces principal components with sparse loadings by introducing an L1 penalty on the component coefficients, formulated as an optimisation problem:

$$\min_{\mathbf{U},\mathbf{V}} \|\mathbf{X} - \mathbf{U}\mathbf{V}^\top\|_F^2 + \alpha\|\mathbf{V}\|_1 \quad \text{s.t.} \quad \|\mathbf{u}_i\|_2 \le 1 \tag{8}$$

The L1 penalty forces most component weights to exactly zero, so each principal component depends on only a small subset of the original features, improving interpretability at a small cost in explained variance. The problem is solved iteratively using coordinate descent.

Complexity: Time $O(ndk \cdot \text{iter})$; Space $O(nk + dk)$.

5 Results

5.1 Correctness Verification

Each implementation is compared against its Scikit-learn counterpart on a shared set of test datasets. Agreement is measured by the mean squared difference between projections (after cor-

recting for sign ambiguity) and the difference in explained variance ratios. Results are presented in Table 1.

Method	Projection distance	Explained variance distance
PCA by SVD	2.648×10^{-15}	9.767×10^{-16}
PCA by Eigendecomposition	8.67×10^{-16}	3.94×10^{-16}
Kernel PCA	7.192×10^{-15}	1.644×10^{-15}
Randomized PCA	0.096979	0.166899
Incremental PCA	3.925×10^{-16}	0.001001

Table 1: Comparison for some of the PCA methods with corresponding Sklearn module.

5.2 Scaling Behaviour

The following plots summarize the mean execution time (Fig. 1) and peak memory (Fig. 2) footprint for the implemented algorithms ($n = 1000, d = 50, k = 5, \sigma = 0.01$):

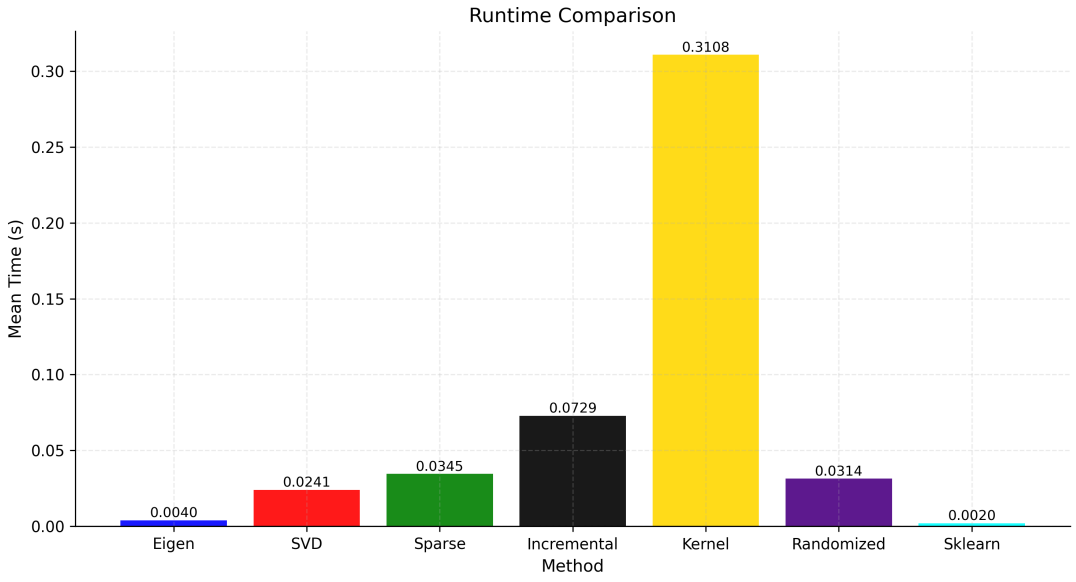


Figure 1: Runtime comparison for all the PCA methods.

For scaling experiments see Appendix, Additional Figures.

Standard Eigendecomposition and Kernel PCA are expected to exhibit cubic scaling in n, becoming impractical at $n \gtrsim 10{,}000$ and $n \gtrsim 5{,}000$ respectively due to memory and time constraints. Randomized and Incremental PCA are expected to scale approximately linearly in n for fixed k. Results confirming these predictions are presented here.

5.3 Numerical Stability

The numerical stability of each algorithm was evaluated for noise levels 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, and 10^{-1}, in terms of projection distance and expected variance ratio distance between the clean and noisy data (see Figures 3- 5).

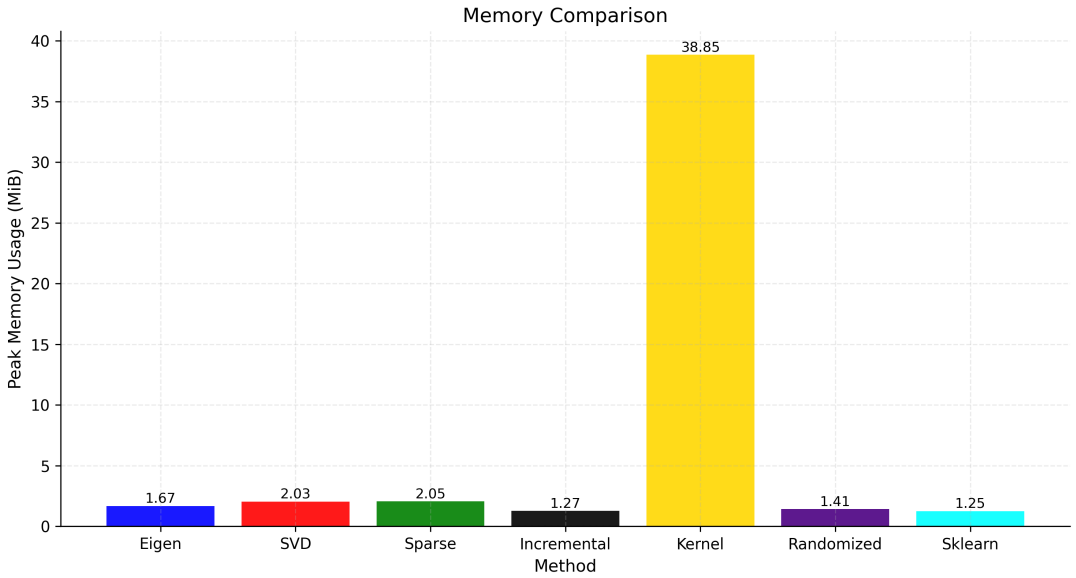


Figure 2: Memory usage comparison for all the PCA methods.

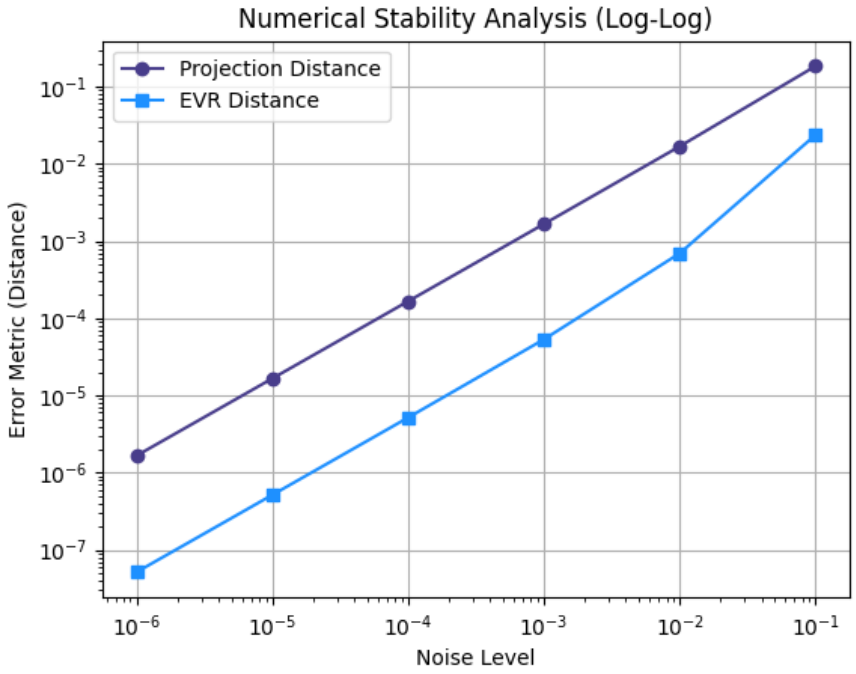


Figure 3: Numerical stability analysis for kernel PCA.

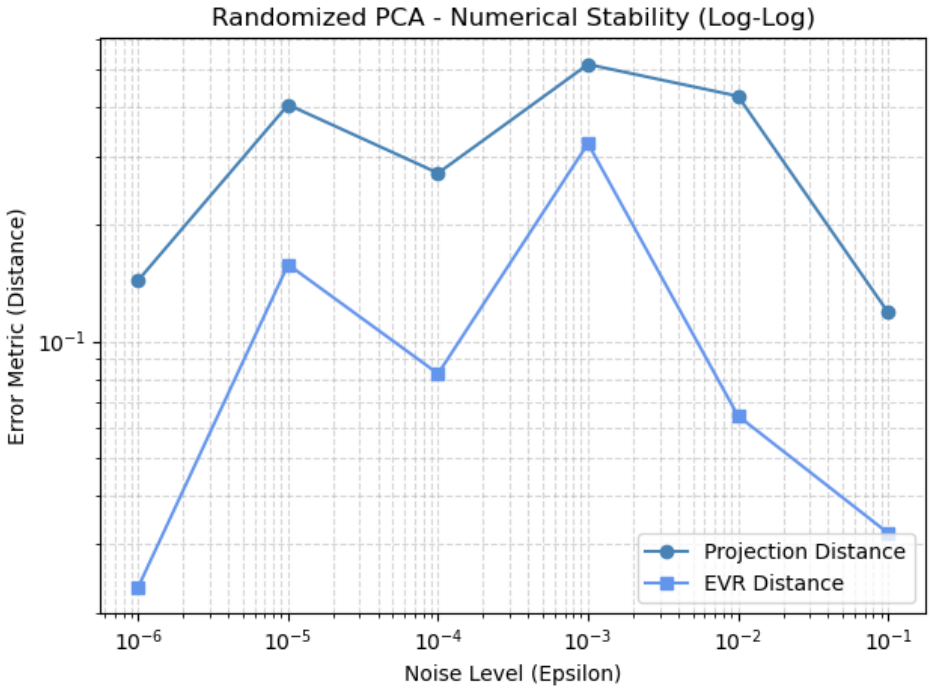


Figure 4: Numerical stability analysis for randomized PCA.

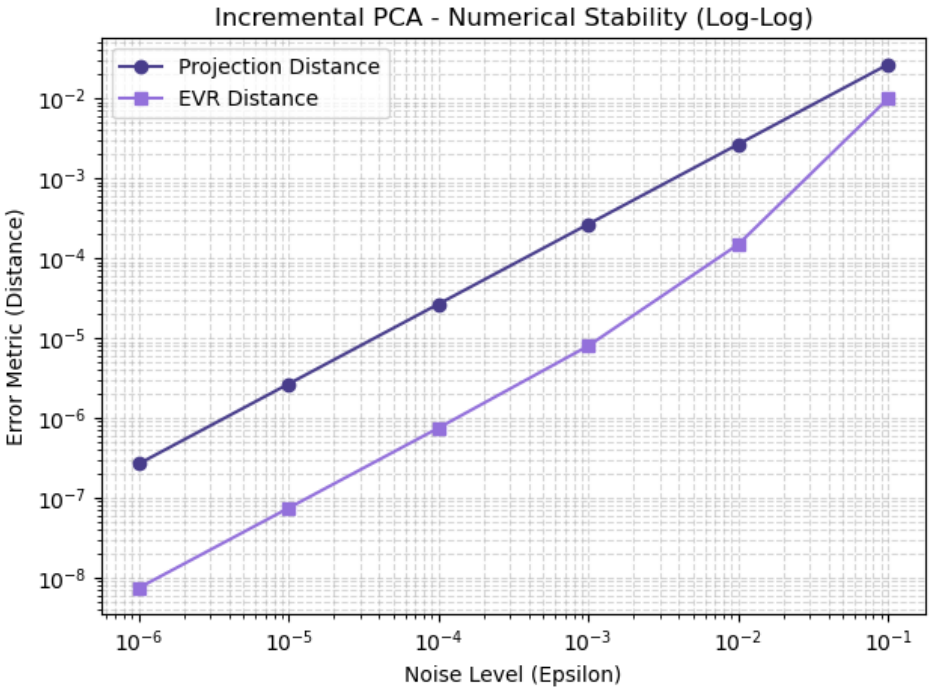


Figure 5: Numerical stability analysis for incremental PCA.

5.4 Impractical Regime

6 Conclusion and Outlook

This report presented a systematic comparison of six PCA implementations across dimensions of correctness, omputational complexity, empirical performance, and umerical stability. The results confirm that no single method dominates across all settings: the choice of PCA algorithm depends strongly on the structure of the data, the available memory, and the number of components required.

For small to medium datasets with $n \ll d$, Eigendecomposition is straightforward and exact. For large datasets with moderate d, SVD offers better numerical stability and similar cost. When only a small number of components are needed from a large dataset, Randomized PCA offers substantial speedups with negligible loss of accuracy. Incremental PCA is the method of choice when data cannot fit in memory or arrives as a stream. Sparse PCA is appropriate when interpretability of components is important and some loss of explained variance is acceptable. Kernel PCA is uniquely suited to non-linear data but requires careful management of its quadratic memory cost (the improved algorithm of Zheng et al. [9] can potentially be employed to significantly extends its practical range).

Future work could explore adaptive selection of the kernel hyperparameter γ for Kernel PCA, integration of the Nyström approximation as a further memory reduction strategy, and extension of the benchmarking framework to real-world high-dimensional datasets in genomics, image processing, and natural language processing.

7 Acknowledgements

The authors thank the teaching staff of DS3294: Data Science Practice for their guidance and for providing the project brief and report template.

8 Author Contributions

For the programming part, the contributions were as follows:

Aurokrupa Sahoo: PCA by Eigendecomposition, PCA by SVD, Sparse PCA implementation, various analyses (correctness, runtime, memory usage, numerical stability and scaling behaviour).

Anshita Singh: Kernel PCA implementation, non-linear data generation.

Debagnik Das: Randomized PCA implementation.

Sadekar Suchet Samir: Incremental PCA implementation, correlated, uncorrelated, and low-rank data generation.

All authors contributed to the comparative review and writing.

9 Code availability

The code for this project is available at the following GitHub repository: https://github.com/Suchet17/PCA_Methods_Comparison.git.

10 References

References

[1] H. Cardot and D. Degras. Online principal component analysis in high dimension: which algorithm to choose? *International Statistical Review*, 86(1):29–50, 2018.

[2] M. Greenacre, P. J. F. Groenen, T. Hastie, A. Iodice D'Enza, A. Markos, and E. Tuzhilina. Principal component analysis. *Nature Reviews Methods Primers*, 2(1):100, 2022. doi: 10.1038/s43586-022-00184-w.

[3] N. Halko, P.-G. Martinsson, and J. A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53 (2):217–288, 2011.

[4] H. Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441, 1933.

[5] K. Pearson. On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11):559–572, 1901.

[6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[7] D. A. Ross, J. Lim, R.-S. Lin, and M.-H. Yang. Incremental learning for robust visual tracking. *International Journal of Computer Vision*, 77(1-3):125–141, 2008.

[8] B. Schölkopf, A. Smola, and K.-R. Müller. Kernel principal component analysis. In *Artificial Neural Networks — ICANN'97*, pages 583–588, Berlin, Heidelberg, 1997. Springer.

[9] W. Zheng, C. Zou, and L. Zhao. An improved algorithm for kernel principal component analysis. *Neural Processing Letters*, 22(1):49–56, 2005. doi: 10.1007/s11063-004-0036-x.

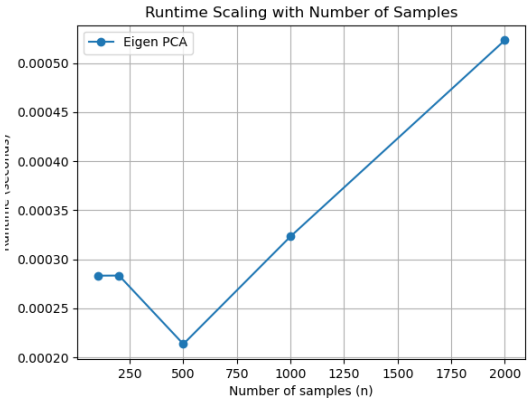
[10] H. Zou, T. Hastie, and R. Tibshirani. Sparse principal component analysis. *Journal of Computational and Graphical Statistics*, 15(2):265–286, 2006.

11 Appendix

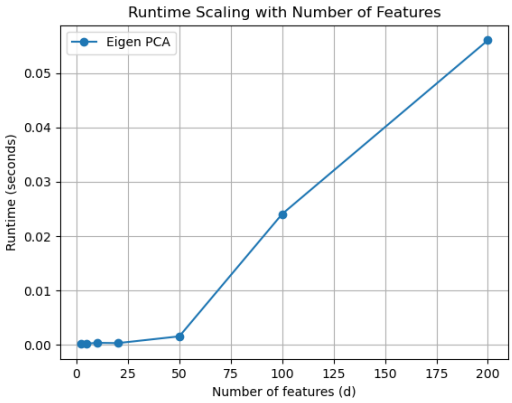
11.1 Additional Figures

Scaling experiments are conducted by varying n with fixed $d = 50$, and varying d with fixed $n = 1{,}000$. Runtime and peak memory usage are recorded for each method and dataset size (see Figures 6- 11).

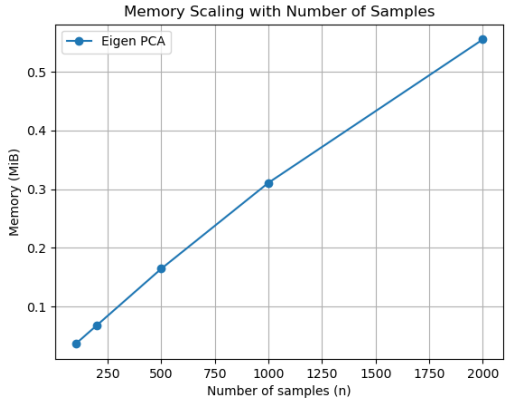
Standard PCA is mathematically restricted to identifying linear structures and fails to resolve observations from the concentric hyperspheres dataset, as the global mean of all shells coincides at the origin. In contrast, Kernel PCA succeeds by utilizing the "kernel trick" to implicitly project the data into a high-dimensional feature space. By performing linear PCA within this kernel space, the algorithm resolves the non-linear relationships, effectively "unrolling" the hyperspheres into a separable low-dimensional representation (see Fig 12).



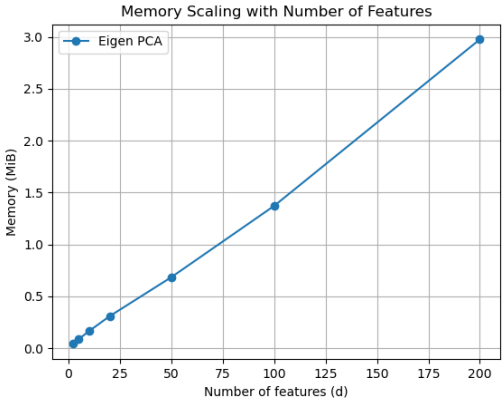
Runtime variation with n.



Runtime variation with d.

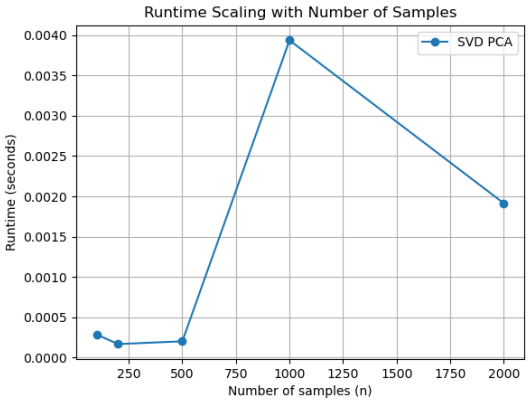


Memory usage variation with n.

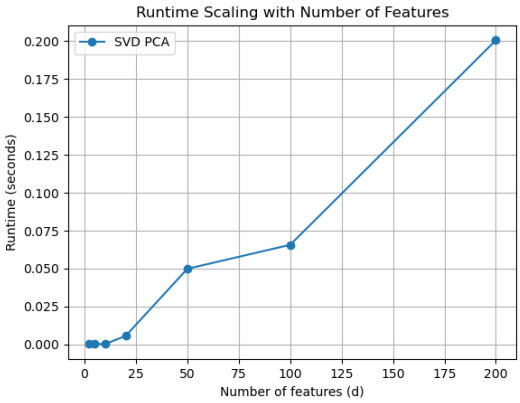


Memory usage variation with d.

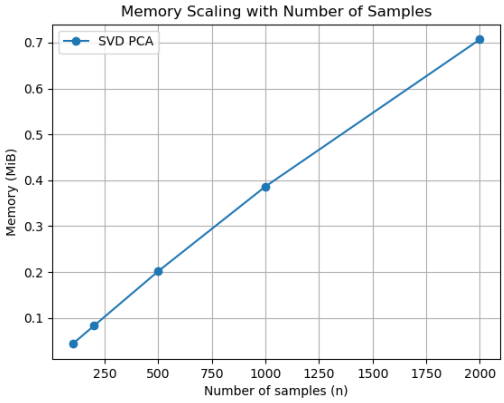
Figure 6: Scaling plots for PCA by Eigendecomposition.



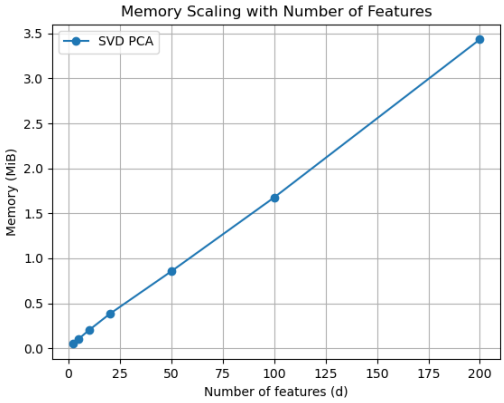
Runtime variation with n.



Runtime variation with d.

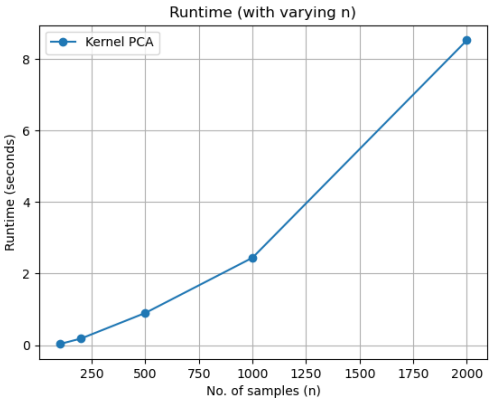


Memory usage variation with n.

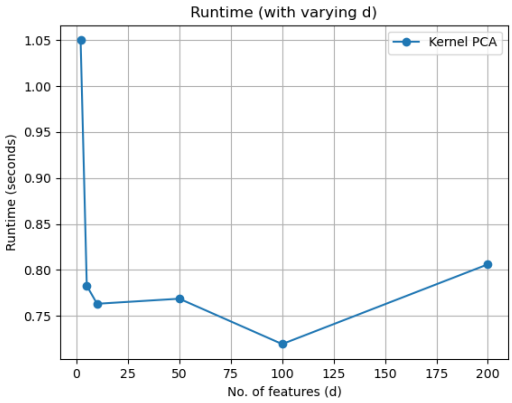


Memory usage variation with d.

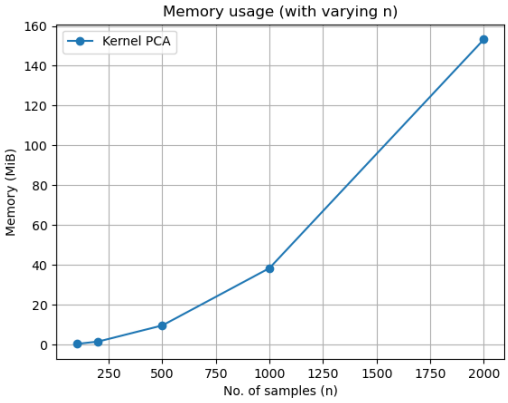
Figure 7: Scaling plots for PCA by SVD.



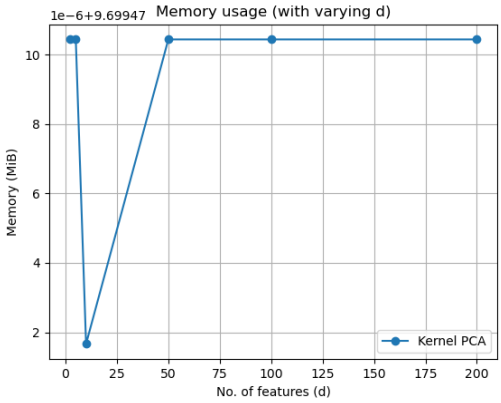
Runtime variation with n.



Runtime variation with d.

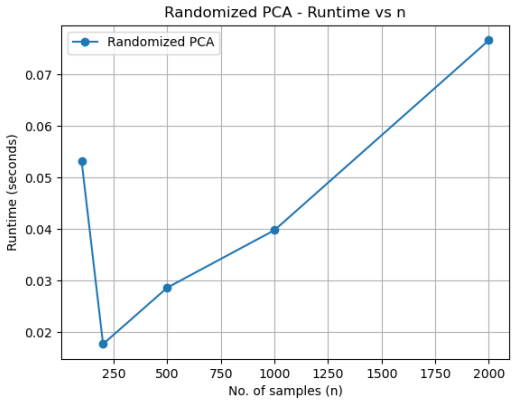


Memory usage variation with n.

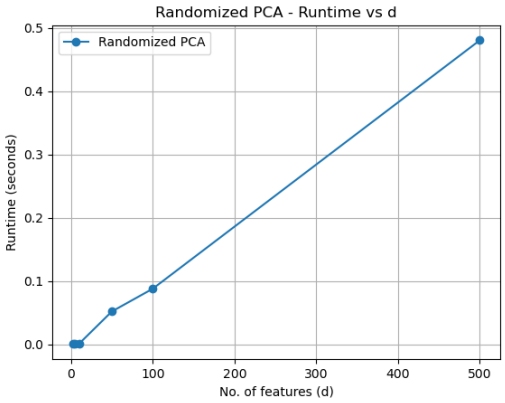


Memory usage variation with d.

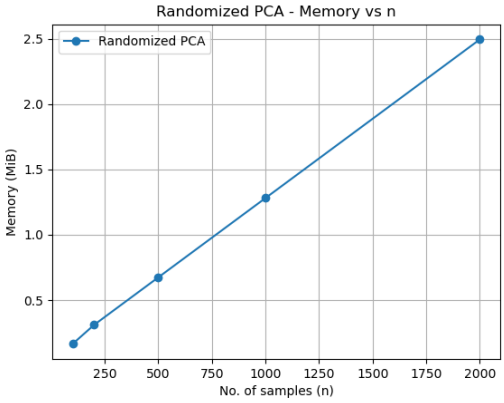
Figure 8: Scaling plots for kernel PCA.



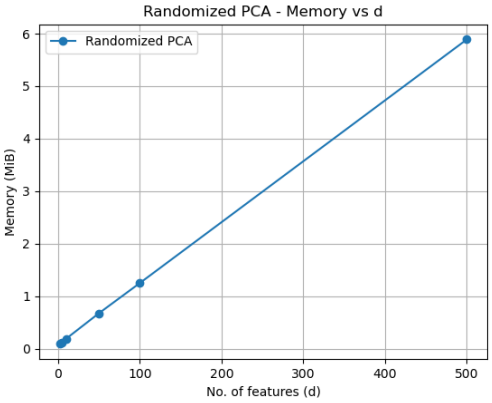
Runtime variation with n.



Runtime variation with d.

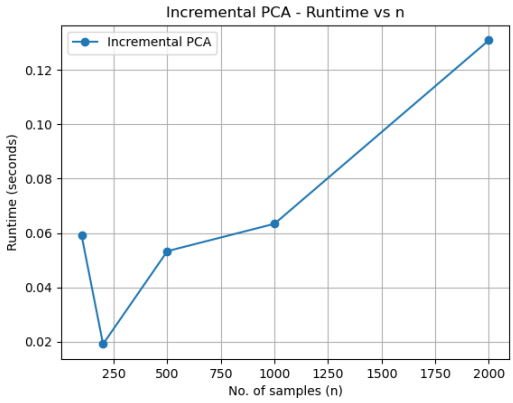


Memory usage variation with n.

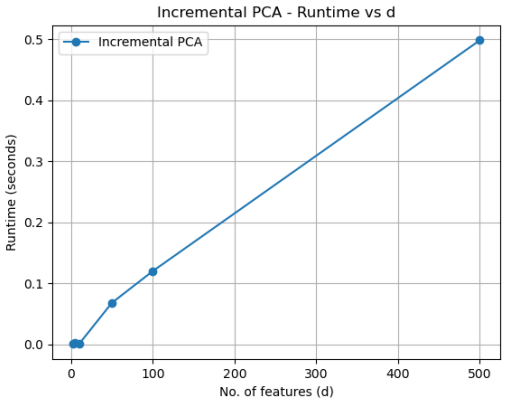


Memory usage variation with d.

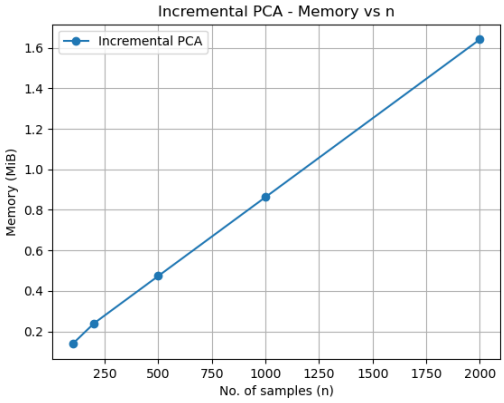
Figure 9: Scaling plots for randomized PCA.



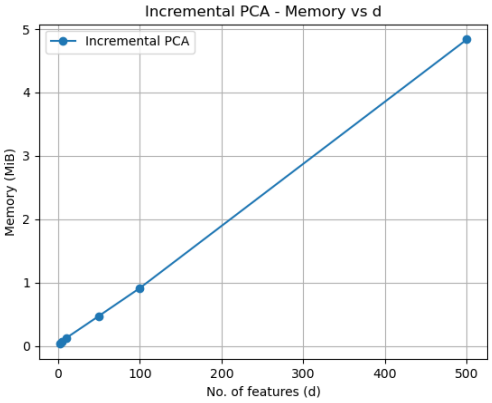
Runtime variation with n.



Runtime variation with d.

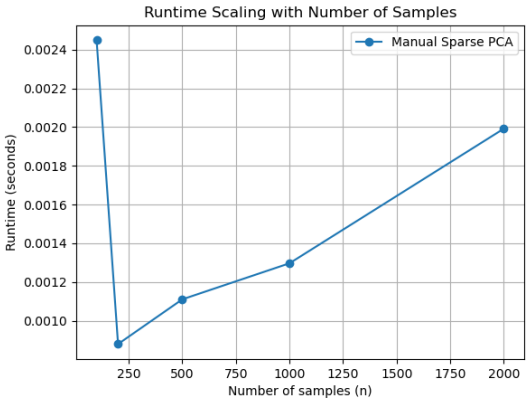


Memory usage variation with n.

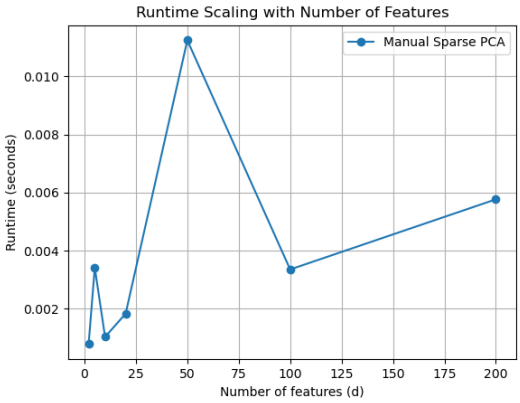


Memory usage variation with d.

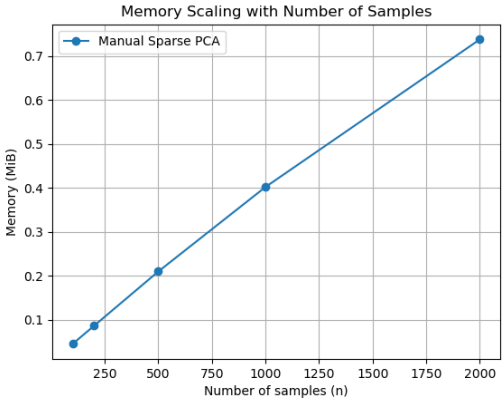
Figure 10: Scaling plots for incremental PCA.



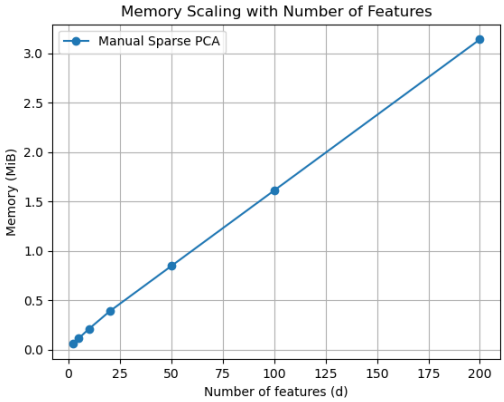
Runtime variation with n.



Runtime variation with d.



Memory usage variation with n.



Memory usage variation with d.

Figure 11: Scaling plots for sparse PCA.

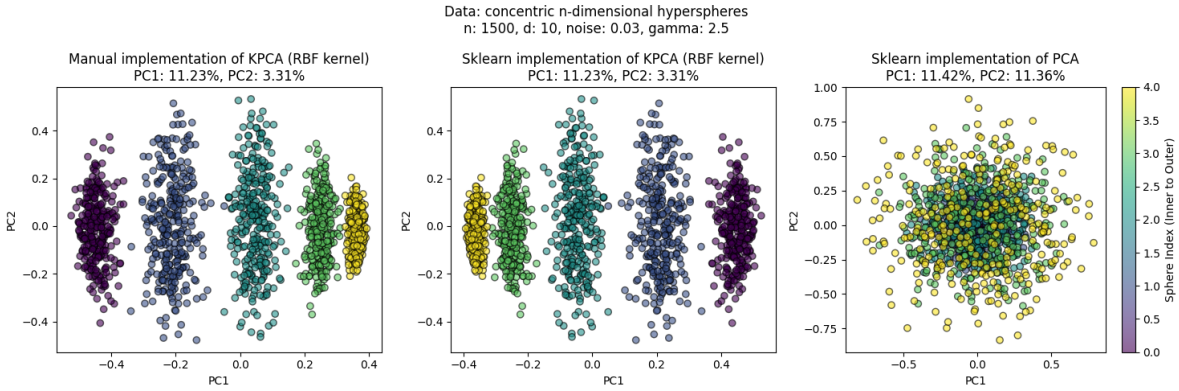


Figure 12: Implementing Kernel PCA and PCA for non-linear data.

12 Algorithms

Algorithm 1 PCA via Eigendecomposition

Require: Data matrix $X \in \mathbb{R}^{n \times d}$, number of components k
Ensure: Reduced data X_{reduced}, principal components V_k
Compute mean μ and center data: $X_c \leftarrow X - \mu$
Compute covariance matrix: $C \leftarrow \frac{1}{n-1} X_c^\top X_c$
Compute eigendecomposition: $CV = \Lambda V$
Sort eigenvalues in descending order
Select top k eigenvectors: V_k
Project data: $X_{\text{reduced}} \leftarrow X_c V_k$

Algorithm 2 PCA via SVD

Require: Data matrix $X \in \mathbb{R}^{n \times d}$, number of components k
Ensure: Reduced data X_{reduced}, principal components V_k
Compute mean μ and center data: $X_c \leftarrow X - \mu$
Compute SVD: $X_c = U \Sigma V^\top$
Select top k right singular vectors: V_k
Project data: $X_{\text{reduced}} \leftarrow X_c V_k$
Compute explained variance: $\lambda_i = \sigma_i^2/(n-1)$

Algorithm 3 Kernel PCA

Require: Data matrix $X \in \mathbb{R}^{N \times d}$, kernel parameter γ, number of components k
Ensure: Projections $\boldsymbol{\alpha} \in \mathbb{R}^{N \times k}$, eigenvalues $\boldsymbol{\lambda}$
Compute $K_{ij} = \exp(-\gamma \|x_i - x_j\|^2)$ for all i, j
Centre: $K_c \leftarrow K - \mathbf{1}_N K - K \mathbf{1}_N + \mathbf{1}_N K \mathbf{1}_N$
Compute eigendecomposition: $K_c \mathbf{V} = \boldsymbol{\Lambda} \mathbf{V}$
Sort eigenpairs by descending eigenvalue; clip negatives to zero
$\boldsymbol{\lambda} \leftarrow$ top k eigenvalues; $\boldsymbol{\alpha} \leftarrow$ corresponding eigenvectors $\times \sqrt{\boldsymbol{\lambda}}$

Algorithm 4 Randomized PCA

Require: Centered data $X_c \in \mathbb{R}^{n \times d}$, components k, oversampling p
Ensure: Approximate principal components and projections
Draw random matrix $\Omega \in \mathbb{R}^{d \times (k+p)}$
Compute sample matrix: $Y \leftarrow X_c \Omega$
Compute orthonormal basis: $Q \leftarrow \text{QR}(Y)$
Project: $B \leftarrow Q^\top X_c$
Compute SVD: $B = \tilde{U} \Sigma V^\top$
Compute approximate components: $U \leftarrow Q \tilde{U}$
Use top k components for projection

Algorithm 5 Incremental PCA

Require: Data matrix X, batch size b, components k
Ensure: Principal components learned incrementally
  Split X into batches of size b
  Initialize mean and components
  for each batch X_i **do**
    Update mean
    Center batch: $X_i \leftarrow X_i - \mu$
    Combine with previous components
    Perform truncated SVD
    Retain top k components
  end for
  Return final components

Algorithm 6 Sparse PCA

Require: Data matrix $X \in \mathbb{R}^{n \times d}$, components k, sparsity parameter α
Ensure: Sparse principal components
  Initialize U, V
  repeat
    Update U with fixed V
    Update V with fixed U using L_1 penalty
  until convergence
  Return sparse components V

12.1 Complexity Summary

Method	**Time Complexity**	**Space Complexity**
Eigendecomposition	$O(nd^2 + d^3)$	$O(d^2)$
SVD	$O(\min(n,d)^2 \cdot \max(n,d))$	$O(nd)$
Kernel PCA	$O(n^3)$	$O(n^2)$
Randomized PCA	$O(ndk)$	$O(nk + dk)$
Incremental PCA	$O(bdk)$ per batch	$O(bd + kd)$
Sparse PCA	$O(ndk \cdot \text{iter})$	$O(nk + dk)$

Table 2: Theoretical time and space complexity of each PCA method. n: number of observations; d: number of features; k: number of components; b: batch size.